

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\workflows\wf_8d68d336-288\agent-a6bf19617b4c7dcc9.jsonl`  
- SHA-256: `f38ad9e472d29c9f9a61b803ab363b25e261ed93b2dd9353a3e376402c12a45e`  
- Source modified: `2026-07-03T19:49:54+00:00`  
- Imported at: `2026-07-08T16:00:15+00:00`  
- project: `wf_8d68d336-288`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-03T19:48:21.202Z)

Repo (my in-progress optimization branch): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/fix-worktree

Key files to Read:

- backend/pipeline/detectors/wasb_infer.py ? the inference loop (DataLoader ? detector.run_tensor ? tracker). Contains the streaming SequentialFrameStore, the _PrefetchIter I just added, build_windows, and the tracker re-run loop.
- backend/pipeline/detectors/native_wasb_runner.py ? builds the wasb_infer.py argv on the GPU box (native runner). Has device/stream_video/batch_size/prefetch_batches config.
- backend/config/models.py (WasbIndexConfig) and backend/config/presets.py (cloud-serving preset).
- deploy/cloudrun/Dockerfile ? the WASB conda env is torch 1.11.0+cu113 (py3.8), a SEPARATE env from the app's torch. The detector shells out to it as a subprocess.

The WASB model itself lives at /opt/models/WASB-SBDT (nttcom/WASB-SBDT) ? you can't read it, but reason from how wasb_infer.py calls build_detector(cfg) / detector.run_tensor(imgs, trans) and build_tracker(cfg).

MEASURED BASELINE (first real L4 cloud run, 2026-07-03): a ~7.5-min 60fps badminton clip took Phase-2 (WASB detection) 727.9s (~0.6x realtime); GPU utilization sat <1% the whole time. Hardware: NVIDIA L4 (Ada sm_89, 23GB), 8 vCPU, 32GB. Streaming mode forces DataLoader num_workers=0 (single-process), so CPU batch assembly (cv2 decode + PIL + ToTensor) serializes with GPU inference.

CRITICAL CONSTRAINT ? the D5/D16 byte-parity guarantee: the repo REQUIRES that the rally/window OUTPUT (the candidate windows and trajectory CSV) stay bit-identical / deterministic across runs and across the disk-frames vs streaming paths. This is asserted by a real parity test. An optimization that changes detector NUMERICS (FP16, TF32, cudnn.benchmark algo selection, NVDEC vs CPU decode) risks changing the output and breaking that guarantee. Batch size and prefetch do NOT change per-frame numerics (same forward, just grouped/overlapped) so they are parity-safe. Already implemented on this branch: indexing.wasb.batch_size + prefetch_batches knobs, the cloud-serving preset opts into batch 64 + prefetch 4.

You are a CUDA/PyTorch performance engineer. Investigate ONE optimization lever in depth by READING the actual code. Lever "pinned-nonblocking": Investigate pinned (page-locked) host memory + non_blocking=True H2D transfers and whether the input tensor is moved to cuda efficiently. In the num_workers=0 streaming path, is there a .to(device) copy that blocks? Would pin_memory=True on the DataLoader (or manual pinning in the prefetch thread) + non_blocking copies overlap transfer with compute? This is parity-SAFE (same bytes, just faster transfer). Locate the H2D copy in run_tensor/wasb_infer and sketch the change.

Read the named files, reason from the real code, and return the structured finding. Be concrete

about code_locations (file:function). Be honest about parity_risk ? the byte-parity guarantee is load-bearing.

2. assistant (2026-07-03T19:48:23.641Z)

I'll investigate the pinned-memory + non-blocking H2D transfer lever by reading the actual code.

3. user (2026-07-03T19:48:26.465Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39         every 1000] [--fresh]
40
41     """
42
43     import argparse
44     import csv
45     import glob
46     import json
47     import logging
48     import os
49     import os.path as osp
50     from collections import defaultdict
51     from contextlib import contextmanager
52
53     logger = logging.getLogger(__name__)
54
55     CACHE_VERSION = 1
56     _DET_FILE = "det_raw.jsonl"
57     _MANIFEST_FILE = "manifest.json"
```

```

55
56
57 def _build_cfg(weights: str, sport: str, device: str = "cuda"):
58     """Compose the WASB eval config via Hydra, overriding model/weights/device.
59
60     ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
61     preserved),
62     ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
63     ``runner.gpus=[0]``;
64     cpu/mps run with no GPU list so the detector places tensors on ``device``.
65     """
66     from hydra import compose, initialize
67
68     gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
69     with initialize(config_path="configs", version_base=None):
70         cfg = compose(
71             config_name="eval",
72             overrides=[
73                 f"dataset={sport}",
74                 "model=wasb",
75                 f"detector.model_path={weights}",
76                 f"runner.device={device}",
77                 f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to
78     this box
79         ],
80     )
81     return cfg
82
83 def _frame_id(path: str) -> int:
84     """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
85     180)."""
86     stem = osp.splitext(osp.basename(path))[0]
87     digits = "".join(ch for ch in stem if ch.isdigit())
88     return int(digits) if digits else -1
89
90 # ----- #
91 # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
92 # ----- #
93
94 def _cache_paths(cache_dir: str):
95     return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
96
97
98 def default_cache_dir(out_csv: str) -> str:
99     """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
100     d = osp.dirname(osp.abspath(out_csv))
101     stem = osp.splitext(osp.basename(out_csv))[0]
102     return osp.join(d, stem + "_wasbcache")
103
104
105 def _atomic_write_text(path: str, text: str) -> None:
106     """Write then rename so a crash mid-write can't leave a half-written file."""
107     tmp = path + ".tmp"
108     with open(tmp, "w") as f:
109         f.write(text)
110         f.flush()
111         os.fsync(f.fileno())
112     os.replace(tmp, path)
113
114
115 def _det_plain(det) -> list:
116     """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
117     list."""
118     xy = det["xy"]

```

```

115         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
116
117
118     def _dets_to_tracker(plain_list):
119         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
120         the tracker does vector math / np.linalg.norm on it)."""
121         import numpy as np
122
123         return [
124             {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
125             for p in plain_list
126         ]
127
128
129     def write_manifest(cache_dir: str, manifest: dict) -> None:
130         _, mpath = _cache_paths(cache_dir)
131         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
132
133
134     def read_manifest(cache_dir: str):
135         _, mpath = _cache_paths(cache_dir)
136         if not osp.exists(mpath):
137             return None
138         try:
139             with open(mpath) as f:
140                 return json.load(f)
141         except (json.JSONDecodeError, OSError):
142             return None
143
144
145     def manifest_compatible(
146         manifest: dict,
147         *,
148         frames_dir: str,
149         weights: str,
150         sport: str,
151         frames_in: int,
152         frames_total: int,
153         device: str = "cuda",
154     ) -> bool:
155         """A cache is reusable only if the run that produced it matches this run's inputs.
156
157         ``device`` defaults to ``cuda`` so caches written before device-keying (no
158         ``device``
159         field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
160         cache
161         (detections can differ numerically across devices), and vice-versa.
162
163         ``frames_dir`` is the **already-canonical cache key** the caller stored in the
164         manifest
165         (``run()`` passes its ``cache_key`` here, and writes the same value into the
166         manifest):
167         ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
168         ``video:<abspath>``
169         key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
170         it here:
171         ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
172         prepended,
173         so a streaming cache would never match itself and every re-run would recompute
174         (issue #348).
175         """
176         if not manifest or manifest.get("version") != CACHE_VERSION:
177             return False
178         return (

```

```

171         manifest.get("frames_dir") == frames_dir
172         and manifest.get("weights") == weights
173         and manifest.get("sport") == sport
174         and manifest.get("frames_in") == frames_in
175         and manifest.get("frames_total") == frames_total
176         and manifest.get("device", "cuda") == device
177     )
178
179
180 def reset_cache(cache_dir: str) -> None:
181     """Drop any existing cache so the next run starts clean."""
182     det_jsonl, mpath = _cache_paths(cache_dir)
183     for p in (det_jsonl, mpath):
184         if osp.exists(p):
185             os.remove(p)
186
187
188 def load_and_clean_cache(cache_dir: str):
189     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
190     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
191     prefix so a trailing half-written line from a crash can't corrupt future appends.
192
193     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
194     [x, y, score, scale] candidates (accumulated across every window the frame is in,
195     in window order ? identical to a non-resumed run).
196     """
197     det_jsonl, _ = _cache_paths(cache_dir)
198     det_by_fid = defaultdict(list)
199     valid: list = []
200     if osp.exists(det_jsonl):
201         with open(det_jsonl, "r") as f:
202             for line in f:
203                 s = line.strip()
204                 if not s:
205                     continue
206                 try:
207                     obj = json.loads(s)
208                 except json.JSONDecodeError:
209                     break # truncated trailing line (crash mid-flush)
210                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
211                     break
212                 valid.append(s)
213                 for fid, plain in obj["f"]:
214                     det_by_fid[int(fid)].extend([list(p) for p in plain])
215                 # Guarantee a clean append boundary by rewriting only the good prefix.
216                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
217     return len(valid), det_by_fid
218
219
220 def _append_windows(fh, objs) -> None:
221     """Append window records as JSONL and durably flush (bounded loss on crash)."""
222     for o in objs:
223         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
224     fh.flush()
225     os.fsync(fh.fileno())
226
227
228 def _atomic_write_trajectory(out_csv: str, rows) -> None:
229     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
230     tmp = out_csv + ".tmp"
231     with open(tmp, "w", newline="") as f:
232         w = csv.writer(f)
233         w.writerow(["Frame", "Visibility", "X", "Y"])
234         w.writerows(rows)
235         f.flush()

```

```

236         os.fsync(f.fileno())
237         os.replace(tmp, out_csv)
238
239
240     # ----- #
241     # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
242     # ----- #
243     _STREAM_FRAME_FMT = "{:08d}.png"
244
245
246     def synthetic_frame_path(index: int) -> str:
247         """Stable, index-encoding frame name used by the streaming path.
248
249         It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
250         downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
251         frames came from disk or from the in-memory stream. The streaming image loader
252         inverts this name back to an index to fetch the decoded frame.
253         """
254         return _STREAM_FRAME_FMT.format(int(index))
255
256
257     def build_windows(frames: list, frames_in: int):
258         """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
259         + checkpoint). Pure list math, identical for the disk and streaming paths.
260
261         Returns a list of windows, each a list of ``frames_in`` consecutive paths
262         (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
263         caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
264         ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
265         """
266         return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
267
268
269     class SequentialFrameStore:
270         """In-memory sliding buffer over a forward-only frame reader, sized for the
271         detector's sliding-window access pattern (peak O(window) frames, not O(video)).
272
273         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
274         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
278         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
279         earliest index any not-yet-finished window can still ask for) and decode each source
280         frame exactly once via the forward-only ``reader(index)``.
281         """
282
283         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
284             self._reader = reader
285             self._total = int(total)
286             self._window = max(1, int(window))
287             self._buf: dict = {}
288             # On a resumed run the detector's first needed frame is `start_index`; begin
289             # pulling there so the reader can skip (seek past) the already-cached prefix
290             # instead of decoding 0..start_index-1 just to throw them away.
291             self._next = int(start_index) # next index not yet pulled from the reader
292             self._max_seen = int(start_index) - 1
293             self._floor = int(start_index) # lowest index we still keep (never evict below)
294
295         def get(self, index: int):
296             index = int(index)
297             if index < self._floor:
298                 raise KeyError(
299                     f"frame {index} already evicted (below floor {self._floor}); the access
300                     "

```

```

300         f"pattern must stay within a window of the highest frame seen"
301     )
302     if index >= self._total:
303         raise KeyError(f"frame {index} out of range (total {self._total})")
304     # Pull forward from the reader until the requested index is buffered.
305     while self._next <= index:
306         self._buf[self._next] = self._reader(self._next)
307         self._next += 1
308     if index > self._max_seen:
309         self._max_seen = index
310     # Evict frames older than the earliest a still-running window could re-request:
311     # once we've seen index `m`, no future window starts before `m - (window-1)`.
312     new_floor = max(self._floor, self._max_seen - (self._window - 1))
313     for k in range(self._floor, new_floor):
314         self._buf.pop(k, None)
315     self._floor = new_floor
316     return self._buf[index]
317
318
319 def _index_from_synthetic_path(path: str) -> int:
320     """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
321
322     Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
323     in lockstep with the on-disk naming scheme.
324     """
325     return _frame_id(path)
326
327
328 def _bgr_to_pil_rgb(frame_bgr):
329     """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
330     EXACT
331     PIL RGB image WASB's disk path produces.
332
333     The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
334     Image.open(PNG).convert('RGB')``;
335     because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
336     COLOR_BGR2RGB)``
337     bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
338     streamed
339     frame indistinguishable from the on-disk one to everything downstream.
340     """
341     import cv2
342     from PIL import Image
343
344     return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
345
346
347 def _make_streamed_read_image(store, real_read_image):
348     """Build the ``read_image`` replacement used during a streaming detector pass.
349
350     A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
351     image,
352     matching ``read_image``'s real return type); any path that doesn't parse to an index
353     falls
354     through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
355     testable
356     without the WSL-only ``dataloaders`` package.
357     """
358
359     def _read_image(path, *args, **kwargs):
360         idx = _index_from_synthetic_path(path)
361         if idx < 0:
362             return real_read_image(path, *args, **kwargs)
363         return _bgr_to_pil_rgb(store.get(idx))
364

```

```

358         return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
366         decoded
367         frames during the detector pass, byte-for-byte as it would over real PNGs.
368
369         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
370         ``cv2.imread``. ``read_image`` (``utils.utils``) does
371         ``Image.open(path).convert('RGB')``,
372         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
373         feed it a
374         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
375         byte-identical to the disk round-trip, and the shim never touches the filesystem so
376         the
377         ``osp.exists`` guard is sidestepped for synthetic stream paths.
378
379         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
380         actually
381         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
382         in that
383         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
384         imported
385         reference).
386
387         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
388         wrapped
389         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
390         on a
391         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
392         The
393         original reader is restored on exit.
394         """
395         from dataloaders import dataset_loader as _dl
396
397         store = SequentialFrameStore(
398             frame_loader, total, window=window, start_index=start_index
399         )
400         real_read_image = _dl.read_image
401         # Rebind read_image in the consuming module for the duration of the detector pass;
402         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
403         B010-clean.)
404         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
405         ignore[assignment]
406         try:
407             yield store
408         finally:
409             _dl.read_image = real_read_image # type: ignore[assignment]
410
411     # ----- #
412     # Inference
413     # ----- #
414     class _PrefetchIter:
415         """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
416         DataLoader).
417
418         In streaming mode the DataLoader must stay single-process (the in-memory frame store
419         +
420         the cv2.imread shim live in this process and cannot be pickled to workers), so the
421         CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?

```

```

409     measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
410     A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
411     release the GIL for the heavy parts, the store's forward-only access pattern is
412     preserved (one consumer thread iterating sequentially), and the bounded queue caps
413     memory at ``depth`` batches. A producer exception is re-raised in the consumer."""
414
415     _END = object()
416
417     def __init__(self, src, depth: int = 2):
418         import queue
419         import threading
420
421         self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
422         self._exc: "BaseException | None" = None
423
424         def _fill() -> None:
425             try:
426                 for item in src:
427                     self._q.put(item)
428             except BaseException as e: # noqa: BLE001 - surfaced in __next__
429                 self._exc = e
430             finally:
431                 self._q.put(self._END)
432
433         self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
434         self._t.start()
435
436     def __iter__(self):
437         return self
438
439     def __next__(self):
440         item = self._q.get()
441         if item is self._END:
442             if self._exc is not None:
443                 raise self._exc
444             raise StopIteration
445         return item
446
447
448     def run(
449         frames,
450         weights: str,
451         out_csv: str,
452         sport: str = "badminton",
453         limit: int = 0,
454         batch_size: int = 8,
455         num_workers: int = 4,
456         log_every_batches: int = 50,
457         cache_dir: "str | None" = None,
458         flush_every: int = 1000,
459         fresh: bool = False,
460         frame_loader=None,
461         source_key: "str | None" = None,
462         device: str = "cuda",
463         prefetch_batches: int = 0,
464     ) -> str:
465         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
466
467         Two equivalent ways to supply pixels (output is bit-identical between them):
468
469         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
470         globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
471         ``cv2.imread``.
472
473         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)

```

```

472     frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
473     pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
474     reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
475     ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
476     to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
477     ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
``num_workers=0``
478     (the in-process shim + buffer can't cross worker processes).
479
480     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
481     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
482     after reboot still matches.
483     """
484     import time
485
486     import torch
487     from dataloaders import build_img_transforms, build_seq_transforms
488     from dataloaders.dataset_loader import ImageDataset
489     from torch.utils.data import DataLoader
490
491     # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
the
492     # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
493     # the mypy false positive where the main env resolves `trackers` to the Roboflow
package
494     # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
495     # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
496     from trackers import build_tracker # type: ignore[attr-defined]
497     from utils import Center
498
499     from detectors import build_detector
500
501     streaming = frame_loader is not None
502
503     cfg = _build_cfg(weights, sport, device)
504     frames_in = int(cfg["model"]["frames_in"])
505     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
507
508     if streaming:
509         # `frames` is already the ordered list of (synthetic) frame paths.
510         if not isinstance(frames, (list, tuple)):
511             raise SystemExit(
512                 "streaming mode requires `frames` to be a list of frame paths"
513             )
514         frames = list(frames)
515         frames_dir = source_key or "stream"
516     else:
517         frames_dir = frames
518         frames = sorted(
519             glob.glob(osp.join(frames_dir, "*.png"))
520             + glob.glob(osp.join(frames_dir, "*.jpg"))
521         )
522     if limit:
523         frames = frames[:limit]
524     if len(frames) < frames_in:
525         where = frames_dir if not streaming else "video stream"
526         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
527
528     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +

```

```

checkpoint).
529     samples = []
530     for window in build_windows(frames, frames_in):
531         annos = [
532             {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
533             for p in window
534         ]
535         samples.append({"frames": window, "annos": annos})
536     windows_total = len(samples)
537
538     # ---- cache setup / resume decision ----- #
539     if cache_dir is None:
540         cache_dir = default_cache_dir(out_csv)
541         os.makedirs(cache_dir, exist_ok=True)
542         det_jsonl, _ = _cache_paths(cache_dir)
543         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
544         manifest = None if fresh else read_manifest(cache_dir)
545         resume = manifest_compatible(
546             manifest or {},
547             frames_dir=cache_key,
548             weights=weights,
549             sport=sport,
550             frames_in=frames_in,
551             frames_total=len(frames),
552             device=device,
553         )
554         if resume:
555             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
556             logger.info(
557                 f"resuming from cache: {windows_done}/{windows_total} windows already
558 detected"
559             )
560         else:
561             if manifest is not None:
562                 logger.info("cache incompatible with this run -> starting fresh")
563                 reset_cache(cache_dir)
564                 windows_done, det_by_fid = 0, defaultdict(list)
565
566     base_manifest = {
567         "version": CACHE_VERSION,
568         "frames_dir": cache_key,
569         "weights": weights,
570         "sport": sport,
571         "frames_in": frames_in,
572         "frames_total": len(frames),
573         "device": device,
574         "windows_total": windows_total,
575         "windows_done": windows_done,
576     }
577     write_manifest(cache_dir, base_manifest)
578
579     # ---- detector pass over the un-cached windows ----- #
580     remaining = samples[windows_done:]
581     if remaining:
582         _, transform_test = build_img_transforms(cfg)
583         try:
584             _, seq_transform_test = build_seq_transforms(cfg)
585         except Exception:
586             seq_transform_test = None
587         ds = ImageDataset(
588             cfg,
589             remaining,
590             input_wh,
591             output_wh,
592             transform=transform_test,

```

```

592         seq_transform=seq_transform_test,
593         is_train=False,
594     )
595     # Streaming forces single-process loading: the in-memory frame store + the
596     # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
597     loader_workers = 0 if streaming else num_workers
598     loader = DataLoader(
599         ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
600     )
601     detector = build_detector(cfg)
602
603     from contextlib import ExitStack
604
605     t0 = time.time()
606     done = windows_done
607     win_idx = windows_done
608     log_every = max(1, log_every_batches)
609     flush_n = max(1, flush_every)
610     pending = []
611     logger.info(
612         f"detecting on {len(remaining)} remaining windows "
613         f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
614         f"source={'video-stream' if streaming else 'frames-dir'})..."
615     )
616     det_f = open(det_jsonl, "a")
617     try:
618         with ExitStack() as stack:
619             stack.enter_context(torch.no_grad())
620             # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
621             # in-memory decoded frame for that synthetic path; every other line of
622             # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
623             # The detector's first needed frame is `windows_done` (window i starts
at
624             # frame i), so prime the store there for a resume.
625             if streaming:
626                 stack.enter_context(
627                     _streaming_read_image_patch(
628                         frame_loader,
629                         len(frames),
630                         window=frames_in,
631                         start_index=windows_done,
632                     )
633                 )
634             # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
635             # a multi-worker DataLoader already overlaps via its worker processes).
636             batches = (
637                 _PrefetchIter(loader, depth=prefetch_batches)
638                 if prefetch_batches and loader_workers == 0
639                 else loader
640             )
641             for bi, batch in enumerate(batches):
642                 imgs, _hms, trans = batch[0], batch[1], batch[2]
643                 img_paths = [
644                     list(t) for t in batch[-1]
645                 ] # [frame_in][batch] -> path
646                 batch_results, _ = detector.run_tensor(imgs, trans)
647                 nb = imgs.shape[0]
648                 for ib in range(nb):
649                     frames_payload = []
650                     for ie in sorted(batch_results[ib].keys()):
651                         p = img_paths[ie][ib]

```

```

652         fid = _frame_id(p)
653         plain = [_det_plain(d) for d in batch_results[ib][ie]]
654         frames_payload.append([fid, plain])
655         det_by_fid[fid].extend(plain)
656         pending.append({"w": win_idx, "f": frames_payload})
657         win_idx += 1
658     done += nb
659     if len(pending) >= flush_n or done >= windows_total:
660         _append_windows(det_f, pending)
661         pending = []
662         base_manifest["windows_done"] = done
663         write_manifest(cache_dir, base_manifest)
664     if (bi + 1) % log_every == 0 or done >= windows_total:
665         el = time.time() - t0
666         new_done = done - windows_done
667         rate = new_done / el if el > 0 else 0.0
668         eta = (windows_total - done) / rate if rate > 0 else 0.0
669         logger.info(
670             f"{done}/{windows_total} "
671             f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
672             f"elapsed {el:.0f}s | ETA {eta:.0f}s"
673         )
674     if pending:
675         _append_windows(det_f, pending)
676         base_manifest["windows_done"] = done
677         write_manifest(cache_dir, base_manifest)
678     finally:
679         det_f.close()
680     logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
681 else:
682     logger.info("detector cache already complete -> tracker-only re-run")
683
684 # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
685 tracker = build_tracker(cfg)
686 tracker.refresh()
687 rows = []
688 for fid in sorted(det_by_fid.keys()):
689     r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
690     rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
691
692 _atomic_write_trajectory(out_csv, rows)
693 visible = sum(1 for _, v, *_ in rows if v)
694 logger.info(f"wrote {len(rows)} rows -> {out_csv}")
695 logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
696 return out_csv
697
698
699 def extract_frames(video_path: str, frames_dir: str) -> str:
700     """Decode a video to PNG frames named by source frame index (00000000.png ...).
701
702     Idempotent: if the dir already holds the expected number of frames (matching the
703     container's reported frame count) we skip re-decoding ? re-extraction after a
704     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
705     extracting with ffmpeg on the host and passing --frames_dir.
706     """
707     import cv2
708
709     os.makedirs(frames_dir, exist_ok=True)
710     cap = cv2.VideoCapture(video_path)
711     if not cap.isOpened():
712         raise SystemExit(f"cannot open video: {video_path}")
713     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
714     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
715     if expected > 0 and existing >= expected:

```

```

716         cap.release()
717         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
718         return frames_dir
719     i = 0
720     while True:
721         ok, frame = cap.read()
722         if not ok:
723             break
724         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
725         i += 1
726     cap.release()
727     logger.info(f"extracted {i} frames -> {frames_dir}")
728     return frames_dir
729
730
731 def _probe_frame_count(video_path: str) -> int:
732     """Container-reported frame count via cv2 (0 if unknown). Used to size the
733     stream."""
734     import cv2
735     cap = cv2.VideoCapture(video_path)
736     if not cap.isOpened():
737         raise SystemExit(f"cannot open video: {video_path}")
738     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
739     cap.release()
740     return n
741
742
743 def make_video_frame_loader(video_path: str):
744     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
745
746     ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
747     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
748     detector's sliding-window order); it reads sequentially and only uses a frame-peek
749     to
750     skip forward when a *resume* starts past the current position. Each ``cap.read()``
751     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
752     would
753     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
754
755     Returns the count hint from the container so the caller can build the frame list;
756     ``release`` closes the capture.
757     """
758     import cv2
759     cap = cv2.VideoCapture(video_path)
760     if not cap.isOpened():
761         raise SystemExit(f"cannot open video: {video_path}")
762     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
763     state = {"pos": 0} # next index cap.read() will return
764
765     def frame_loader(index: int):
766         index = int(index)
767         if index < state["pos"]:
768             raise RuntimeError(
769                 f"streaming decode is forward-only; got index {index} after
770                 {state['pos']}")
771         if index > state["pos"]:
772             # Forward skip only happens when resuming past cached windows. Seek once.
773             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
774             state["pos"] = index
775         ok, frame = cap.read()
776         if not ok:
777             raise RuntimeError(f"failed to decode frame {index} from {video_path}")

```

```

777         state["pos"] += 1
778         return frame
779
780     def release():
781         cap.release()
782
783     return frame_loader, count_hint, release
784
785
786 def run_video_streaming(
787     video_path: str,
788     weights: str,
789     out_csv: str,
790     sport: str = "badminton",
791     limit: int = 0,
792     batch_size: int = 8,
793     log_every_batches: int = 50,
794     cache_dir: "str | None" = None,
795     flush_every: int = 1000,
796     fresh: bool = False,
797     device: str = "cuda",
798     prefetch_batches: int = 0,
799 ) -> str:
800     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
801     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
802     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
803     ``run``'s docstring for why.
804     """
805     frame_loader, count_hint, release = make_video_frame_loader(video_path)
806     try:
807         if count_hint <= 0:
808             raise SystemExit(
809                 f"could not determine frame count for {video_path}; cannot stream "
810                 f"(fall back to frame extraction with --frames_dir)"
811             )
812         frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
813         source_key = "video:" + osp.abspath(video_path)
814         return run(
815             frame_list,
816             weights,
817             out_csv,
818             sport,
819             limit,
820             batch_size=batch_size,
821             num_workers=0,
822             log_every_batches=log_every_batches,
823             cache_dir=cache_dir,
824             flush_every=flush_every,
825             fresh=fresh,
826             frame_loader=frame_loader,
827             source_key=source_key,
828             device=device,
829             prefetch_batches=prefetch_batches,
830         )
831     finally:
832         release()
833
834
835 def main():
836     ap = argparse.ArgumentParser()
837     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
838     ap.add_argument(
839         "--video", help="video file; frames are extracted internally (in WSL)"
840     )
841     ap.add_argument(

```

```

842         "--frames_out_dir", help="where to extract frames when --video is used"
843     )
844     ap.add_argument(
845         "--stream-video",
846         action="store_true",
847         help="FAST PATH: decode --video on the fly and feed frames straight to "
848         "the detector (no PNG extraction). Output-equivalent to the disk path.",
849     )
850     ap.add_argument("--weights", required=True)
851     ap.add_argument("--out", required=True)
852     ap.add_argument("--sport", default="badminton")
853     ap.add_argument(
854         "--device",
855         default="cuda",
856         choices=["cuda", "mps", "cpu"],
857         help="torch device for the detector (default cuda = the WSL parity path)",
858     )
859     ap.add_argument(
860         "--limit", type=int, default=0, help="cap #frames (for quick tests)"
861     )
862     ap.add_argument("--batch-size", type=int, default=8)
863     ap.add_argument("--num-workers", type=int, default=4)
864     ap.add_argument(
865         "--prefetch-batches",
866         type=int,
867         default=0,
868         help="streaming mode: overlap CPU batch assembly with GPU inference via a "
869         "bounded producer thread of this queue depth (0 = off, the parity default)",
870     )
871     ap.add_argument(
872         "--log-every-batches", type=int, default=50, help="progress log cadence"
873     )
874     ap.add_argument(
875         "--cache-dir",
876         default=None,
877         help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
878     )
879     ap.add_argument(
880         "--flush-every",
881         type=int,
882         default=1000,
883         help="fsync the detector cache every N windows (bounds reboot loss)",
884     )
885     ap.add_argument(
886         "--fresh", action="store_true", help="ignore any existing cache and start over"
887     )
888     args = ap.parse_args()
889
890     if args.stream_video:
891         if not args.video:
892             raise SystemExit("--stream-video requires --video")
893         run_video_streaming(
894             args.video,
895             args.weights,
896             args.out,
897             args.sport,
898             args.limit,
899             batch_size=args.batch_size,
900             log_every_batches=args.log_every_batches,
901             cache_dir=args.cache_dir,
902             flush_every=args.flush_every,
903             fresh=args.fresh,
904             device=args.device,
905             prefetch_batches=args.prefetch_batches,
906         )

```

```

907         return
908
909     frames_dir = args.frames_dir
910     if args.video:
911         frames_dir = extract_frames(
912             args.video, args.frames_out_dir or (args.video + "_frames")
913         )
914     if not frames_dir:
915         raise SystemExit("provide --frames_dir or --video")
916     run(
917         frames_dir,
918         args.weights,
919         args.out,
920         args.sport,
921         args.limit,
922         batch_size=args.batch_size,
923         num_workers=args.num_workers,
924         log_every_batches=args.log_every_batches,
925         cache_dir=args.cache_dir,
926         flush_every=args.flush_every,
927         fresh=args.fresh,
928         device=args.device,
929     )
930
931
932 if __name__ == "__main__":
933     main()
934

```

4. user (2026-07-03T19:48:26.922Z)

```

1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8      see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16     * weights / repo / python / scratch come from the environment first
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24     (``trajectory_hybrid.resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner

```

```

27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     ""
36
37     import logging
38     import os
39     import shutil
40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63
64     @dataclass
65     class NativeWasbConfig:
66         """Resolved settings for a Linux-native WASB run.
67
68         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
69         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
70         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
71         ""
72
73         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
74         python_bin: str = (
75             "python" # the `wasb` conda env python (invoked by path, no activate)
76         )
77         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
78         sport: str = "badminton"
79         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
81         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)

```

```

82     keep_frames: bool = (
83         False # retain the decoded frame cache after success (debug only)
84     )
85     # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
86     # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
87     # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
88     stream_video: bool = True
89     # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
90     # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
91     batch_size: int = 0
92     prefetch_batches: int = 0
93
94     @classmethod
95     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
96         if isinstance(idx_cfg, dict):
97             idx_cfg = IndexingConfig(**idx_cfg)
98         w = idx_cfg.wasb
99         env = os.environ.get
100         # stream_video is tri-state in config: None/absent => native default (stream);
an
101         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
102         sv = w.stream_video
103         return cls(
104             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
105             python_bin=env("WASB_PYTHON") or w.python_bin,
106             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
107             sport=w.sport,
108             device=env("WASB_DEVICE") or w.device,
109             scratch_dir=env("WASB_SCRATCH")
110             or env("RALLY_SCRATCH")
111             or env("TMPDIR")
112             or "",
113             timeout_sec=w.timeout_sec,
114             keep_frames=w.keep_frames,
115             stream_video=(True if sv is None else bool(sv)),
116             batch_size=int(w.batch_size or 0),
117             prefetch_batches=int(w.prefetch_batches or 0),
118         )
119
120
121     class NativeWasbRunner(DetectorRunner):
122         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
123
124         def __init__(self, cfg: NativeWasbConfig):
125             self.cfg = cfg
126             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
127             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
128             self._cuda_available: Optional[bool] = None
129
130             # --- low-level native exec (the single place tests patch)
-----
131         def _run(
132             self, argv: List[str], cwd: Optional[str] = None
133         ) -> subprocess.CompletedProcess:
134             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
135             return subprocess.run(
136                 argv,
137                 cwd=cwd,

```

```

138         capture_output=True,
139         text=True,
140         timeout=(self.cfg.timeout_sec or None),
141     )
142
143     # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
144     def _probe_cuda(self) -> bool:
145         """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
146         (missing python / timeout / import error) is treated as 'no CUDA'."""
147         try:
148             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
149         except (FileNotFoundError, subprocess.TimeoutExpired):
150             return False
151         return res.returncode == 0 and "cuda True" in (res.stdout or "")
152
153     def _cuda_is_available(self) -> bool:
154         """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
155         if self._cuda_available is None:
156             self._cuda_available = self._probe_cuda()
157         return self._cuda_available
158
159     def _effective_device(self) -> str:
160         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
161         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162         if self.cfg.device == AUTO:
163             return DeviceContext(AUTO, self._cuda_is_available()).effective
164         return self.cfg.device
165
166     def _repo_src(self) -> str:
167         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168
169     def _copy_infer_script(self) -> bool:
170         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171         repo_src = self._repo_src()
172         try:
173             os.makedirs(repo_src, exist_ok=True)
174             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
175             return True
176         except OSError as e:
177             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178             return False
179
180     def _rm_rf(self, path: Optional[str]) -> None:
181         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
182         if path:
183             shutil.rmtree(path, ignore_errors=True)
184
185     def healthcheck(self) -> Tuple[bool, str]:
186         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
187         if not self.cfg.weights_path:
188             return False, (
189                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
190             )
191         weights = os.path.expanduser(self.cfg.weights_path)
192         if not os.path.exists(weights):
193             return False, f"WASB weights not found at {weights}"

```

```

194         repo_src = self._repo_src()
195         if not os.path.isdir(repo_src):
196             return False, (
197                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
198                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
199         )
200         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
201         # argparse rc=2 from wasb_infer.py at run time.
202         if self.cfg.device not in VALID_DEVICES:
203             return False, (
204                 f"unknown device {self.cfg.device!r} ? valid: {'',
205                 '.join(VALID_DEVICES)}')."
206             )
207         try:
208             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
209         except FileNotFoundError:
210             return (
211                 False,
212                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
213             )
214         except subprocess.TimeoutExpired:
215             return False, "native torch healthcheck timed out."
216         out = (res.stdout or "").strip()
217         if res.returncode != 0:
218             return (
219                 False,
220                 f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
221                 out).strip()}",
222             )
223         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
224         subprocess).
225         self._cuda_available = "cuda True" in out
226         ctx = DeviceContext(self.cfg.device, self._cuda_available)
227         if ctx.cuda_required_but_missing:
228             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
229             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
230             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
231         if ctx.fell_back:
232             logger.warning(
233                 "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
234                 "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
235                 out,
236             )
237         return True, out
238
239     def run_predict(
240         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
241     ) -> Optional[str]:
242         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
243         None.
244
245         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
246         WSL
247         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
248         the
249         same downstream contract, so the parity comparison is apples-to-apples.
250         """
251         output_dir = os.path.abspath(output_win_dir)
252         os.makedirs(output_dir, exist_ok=True)
253         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
254         norm = str(video_win_path).replace("\\", "/")
255         stem, _ext = os.path.splitext(os.path.basename(norm))

```

```

250         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
251         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
252         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
253
254         if not self._copy_infer_script():
255             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
256             return None
257
258         weights = os.path.expanduser(self.cfg.weights_path)
259         argv = [
260             self.cfg.python_bin,
261             "wasb_infer.py",
262             "--video",
263             os.path.abspath(str(video_win_path)),
264         ]
265         frames_dir: Optional[str] = None
266         if self.cfg.stream_video:
267             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
268             argv.append("--stream-video")
269         else:
270             scratch = self.cfg.scratch_dir or output_dir
271             frames_dir = os.path.join(scratch, f"{key}_frames")
272             argv += ["--frames_out_dir", frames_dir]
273             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
274             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
275             device = self._effective_device()
276             argv += [
277                 "--weights",
278                 weights,
279                 "--sport",
280                 self.cfg.sport,
281                 "--device",
282                 device,
283                 "--out",
284                 out_csv,
285             ]
286             # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
287             # byte-parity guarantee) is unchanged when the knobs are unset.
288             if self.cfg.batch_size:
289                 argv += ["--batch-size", str(self.cfg.batch_size)]
290             if self.cfg.prefetch_batches:
291                 argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
292
293             logger.info(
294                 "Running native WASB inference (device=%s%s) on %s ...",
295                 device,
296                 " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
297                 os.path.basename(norm),
298             )
299             try:
300                 res = self._run(argv, cwd=self._repo_src())
301             except FileNotFoundError:
302                 logger.error(
303                     "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
304                 )
305                 return None
306             except subprocess.TimeoutExpired:
307                 logger.error(
308                     "WASB native inference timed out after %ss.", self.cfg.timeout_sec
309                 )

```

```

310         return None
311     if res.returncode != 0:
312         logger.error(
313             "WASB native inference failed:\n%s",
314             (res.stderr or res.stdout or "")[-2000:],
315         )
316         return None
317     if not os.path.exists(out_csv):
318         logger.error(
319             "WASB native finished but expected CSV not found at %s", out_csv
320         )
321         return None
322     logger.info("WASB trajectory CSV: %s", out_csv)
323
324     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
325     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
326     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
327     if not self.cfg.keep_frames and frames_dir:
328         logger.info(
329             "Cache hygiene: removing frame cache for %s "
330             "(set indexing.wasb.keep_frames=true to retain).",
331             key,
332         )
333         self._rm_rf(frames_dir)
334     return out_csv
335
336     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
337     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
338     trajectory_to_action_windows = staticmethod(
339         TrackNetRunner.trajectory_to_action_windows
340     )
341

```

5. assistant (2026-07-03T19:48:32.747Z)

I have what I need from the two named files. The critical H2D copy is inside WASB's `detector.run\_tensor(imgs, trans)` (in the `/opt/models/WASB-SBDT` repo, which I can't read), but I can reason precisely from how `wasb\_infer.py` calls it and how the DataLoader is constructed. Let me confirm there's no `pin\_memory` anywhere in the branch and check the config knobs.

6. user (2026-07-03T19:48:36.304Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\person_detector.py:12: ``trackers.ByteTrackTracker`` with its
constructor params pinned to ``sv.ByteTrack``'s
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\person_detector.py:103:             self.model.to(self.device)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\person_detector.py:112:             The constructor params are
pinned to ``sv.ByteTrack``'s behaviour so the migration
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\person_detector.py:150:
torch.from_numpy(rgb).permute(2, 0, 1).float().div(255.0).to(self.device)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\rally_gate.py:45:     ``< ~20`` items pinned ``hi`` to the

```

second-to-last sample. On large arrays

7. user (2026-07-03T19:48:38.326Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\models.py:112:    stream_video: Optional[bool] = None
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\models.py:118:    batch_size: Optional[int] = None
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\models.py:121:    prefetch_batches: Optional[int] = None
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\backend\config\presets.py:60:    "wasb": {"batch_size": 64, "prefetch_batches": 4},
```

8. assistant (2026-07-03T19:48:42.761Z)

Confirmed: no `pin\_memory`, no `non\_blocking`, no manual `.pin\_memory()` anywhere in the WASB path. The DataLoader at wasb_infer.py:598 is built with `pin\_memory` unset (defaults to `False`), and the H2D copy lives inside WASB's `detector.run\_tensor(imgs, trans)` which I can't read but can reason about. Let me read the config model and preset context to be precise about where a new knob would land.

9. user (2026-07-03T19:48:44.327Z)

```
95         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98         keep_frames: bool = False
99         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100        # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101        # 0 = no extra floor (the headroom guard still applies).
102        min_free_gb: float = 0.0
103        # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104        # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105        # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106        # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107        # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108        # Tri-state:
109        #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110        #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111        #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112        stream_video: Optional[bool] = None
113        # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114        # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115        # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116        # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
```

```

117     # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118     batch_size: Optional[int] = None
119     # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120     # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121     prefetch_batches: Optional[int] = None
122
123
124     class FusionConfig(BaseModel):
125         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
126         P2).
127
128         Every default reproduces control behaviour: the fusion segmenter persists the
129         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
130         is
131         true ? the person-cue features, but it does NOT gate the served handoff unless a
132         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
133         mask
134         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
135         only);
136         supplied ? off-court persons (spectators / adjacent courts) are excluded.
137         """

```

10. user (2026-07-03T19:48:45.811Z)

```

45         "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46         # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
47         box).
48         # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
49         "indexing": {"skip_ai_handoff": True},
50         "storage": {"backend": "local"},
51     },
52     "cloud-serving": {
53         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
54         # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
55         (D11).
56         # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
57         L4
58         # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
59         # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
60         unchanged.
61         "indexing": {
62             "detector_impl": "native",
63             "skip_ai_handoff": False,
64             "wasb": {"batch_size": 64, "prefetch_batches": 4},
65         },
66         "storage": {"backend": "gcs"},
67     },
68     "cpu-mock": {
69         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
70         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
71         profile.
72         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
73         "storage": {"backend": "local"},
74     },
75 }
76
77 def is_known_profile(profile: str) -> bool:
78     """True for a non-empty, recognised profile name."""

```

11. assistant (2026-07-03T19:49:02.371Z)

I now have the full picture. Let me record my analysis and return the structured finding.

Key facts established from the real code:

1. **The H2D copy is inside `detector.run_tensor(imgs, trans)` (wasb_infer.py:646), which lives in the WASB-SBDT repo. WASB's detector wrapper moves the CPU tensor `imgs` (assembled by the DataLoader from `ToTensor`, so a plain pageable-memory `torch.FloatTensor`) to CUDA before the forward. Standard WASB code does `imgs = imgs.to(self._device)` or `.cuda()` with the default `non_blocking=False`.**
2. **The DataLoader is built WITHOUT `pin_memory` (wasb_infer.py:598-600) ? it defaults to `False`. So the batch tensor sits in ordinary pageable host memory.**
3. **A pageable?device copy is inherently *synchronous*: `cudaMemcpy` from non-pinned memory blocks the host thread and cannot be issued async on a stream, so it serializes with everything. But the copy is small: input is `frames_in` × 3 × `inp_h` × `inp_w` per sample × batch ? for badminton WASB that's ~a few MB per batch of 64. At ~6 GB/s pageable bandwidth that's on the order of low single-digit milliseconds per batch.**
4. **This is NOT the dominant bottleneck. The measured <1% GPU util + 0.6x realtime is CPU-feed-bound (cv2 decode + PIL + ToTensor serializing with GPU), which is exactly what the `_PrefetchIter` thread and larger batch already target. Pinning shaves H2D but the feed thread is the real lever.**
5. **Parity: pinned memory + `non_blocking=True` copy the exact same bytes ? no numeric change. `pin_memory=True` on the DataLoader is 100% parity-safe. The subtlety: `non_blocking=True` from a *pinned* buffer is safe here because the very next op is the model forward on the same default stream, which orders correctly. It is only unsafe if you read the tensor on the host or overwrite the source buffer before the copy completes ? neither happens here.**
6. **The `non_blocking=True` half of the win cannot be applied from this file ? the `.to(cuda)` is inside WASB's `run_tensor`, which we don't edit (editing the vendored repo breaks the "same verified core" contract). So the realizable change from wasb_infer.py alone is `pin_memory=True`, which makes the *existing blocking* copy faster (pinned pageable bandwidth ~2x) but still blocking. Full overlap needs the `non_blocking` copy inside `run_tensor`.**

12. user (2026-07-03T19:49:26.941Z)

`<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.`

You sent (first 200 of 4422 bytes): {"lever": "pinned-nonblocking: set pin_memory=True on the streaming DataLoader (parity-safe, applyable here); non_blocking=True H2D lives inside WASB's run_tensor and is NOT reachable from this repo",
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

13. assistant (2026-07-03T19:49:29.307Z)

I'll pass the fields as proper structured arguments rather than a raw JSON blob.

14. user (2026-07-03T19:49:54.052Z)

Structured output provided successfully